

Agentic RAG implementation with Web based loader

```
!pip install langgraph
!pip install langchain-google-genai
!pip install langchain-community
!pip install langchain-chroma
!pip install langchain-huggingface
!pip install beautifulsoup4
```

Show hidden output

```
import warnings
warnings.filterwarnings('ignore')

# Re-install langchain-huggingface as its previous installation was interrupted
!pip install langchain-huggingface
```

Show hidden output

```
#import libraries
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
```

```
#import models and store embeddings into chroma db
from google.colab import userdata
llm = ChatGoogleGenerativeAI(model="gemini-3.6-flash", temperature=0, api_key=userdata.get("GEMINI"))

urls = [
    "https://en.wikipedia.org/wiki/Artificial_intelligence",
    "https://en.wikipedia.org/wiki/Machine_learning"]

#Web RAG

docs = WebBaseLoader(web_paths=urls).load()
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50).split_documents(docs)
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = Chroma.from_documents(documents=text_splitter, embedding=embeddings)

retriever = vectorstore.as_retriever(search_kwargs={"k":4})
```

Show hidden output

```
#Agentic RAG
class State(TypedDict):
    question:str
    context:str
    answer:str

def agent(state):
    r = llm.invoke(
        f"""Does this requires answer from the web?
        Question:{state['question']}
        Answer only YES or NO."""
    )
    llm_response_content = r.content

    # Ensure llm_response_content is a string. If it's a list, convert to string.
    if isinstance(llm_response_content, list):
        llm_response_str = " ".join(map(str, llm_response_content))
    else:
        llm_response_str = str(llm_response_content) # Ensure it's a string

    # Normalize the response to extract 'YES' or 'NO'
    if 'YES' in llm_response_str.upper():
        extracted_answer = "YES"
    elif 'NO' in llm_response_str.upper():
        extracted_answer = "NO"
```

```

else:
    # Default to 'NO' if the response is ambiguous or unexpected
    extracted_answer = "NO"

return {**state, "context": extracted_answer}

def retrieve(state):
    docs = retriever.invoke(state["question"])
    context = "\n\n".join(doc.page_content for doc in docs)
    return {**state, "context": context}

def generate(state):
    r = llm.invoke(
        f"""Answer using the context.

Question: {state['question']}

Context:
{state['context']}

If the context is insufficient, say you don't have enough information."""
    )
    return {**state, "answer": r.content}

```

```

#Build Graph
graph = StateGraph(State)

graph.add_node("agent",agent)
graph.add_node("retrieve",retrieve)
graph.add_node("generate",generate)

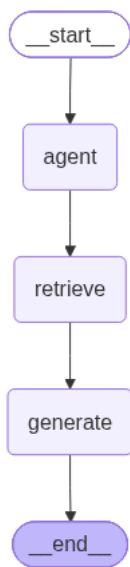
graph.add_edge(START,"agent")
graph.add_edge("agent","retrieve")
graph.add_edge("retrieve","generate")

graph.set_entry_point("agent")

graph.add_conditional_edges(
    "agent",
    lambda s: "retrieve" if s["context"].upper() == "YES" else "generate"
)
graph.add_edge("retrieve","generate")
graph.add_edge("generate",END)

app = graph.compile()
app

```



```

#invoke
result = app.invoke({
    "question": "What is artificial intelligence?",
    "context": "",
    "answer": ""
})

```

```
}}
```

```
print(result["answer"])
```